Shivram Nekkanti

Boston, MA | 617-388-1099 | shivramnekkanti8@gmail.com | linkedin.com/in/shivram08 | github.com/Shivram08

AI engineer with insurance experience across underwriting and loss: statistical modeling, LLM document extraction, and evaluation

Education

Northeastern University — Boston, MA
Master of Science in Data Science — *Expected Dec 2026*
- **Coursework:** Machine Learning, Statistical Modeling, Deep Learning, NLP, MLOps, Database Management

SRM Institute of Science and Technology — India
B.Tech in Computer Science and Engineering (AI/ML specialization) — *May 2024*

Experience

FM Global — Johnston, RI
Data Scientist Intern, Advanced Analytics — *May 2026 – Aug 2026*
- Reduced loss-prediction **MAE 12.5%** with **GLMs** for commercial Fire and Boiler & Machinery losses via feature engineering and missing-data imputation; compared occupancy slices and built separate 100- and 500-year models, not one pooled fit
- Built the building-level dataset in **Databricks** (PostgreSQL federated into Unity Catalog), using **Spark SQL** joins, CTEs, and window functions across 7 tables (4M+ rows) to resolve one-to-many inspection/loss histories into a 300K-row modeling table
- Handled a 93/5/2 class imbalance with SMOTE and PR-AUC-driven threshold selection, setting cutoffs from the relative cost of missing a high-loss building versus over-flagging one instead of defaulting to 0.5
- Co-built a **Tableau** dashboard of MAE gains in loss dollars on 500 high-TIV sites, presented to SMEs and non-technical colleagues
- Fine-tuned **Gemma 3 27B** with LoRA on 10K+ risk reports against the production prompting system, raising BERTScore vs. human-authored reports from **0.78 to 0.91**; a Markdown-AST structure metric exposed template over-memorization
- Removed the RAG layer after benchmarking it against long-context ingestion of each report's source documents: every report's sources fit the model window, so dropping retrieval eliminated retrieval misses and a component to maintain
- Rewrote the engineer-notes parsing stage of the team's **Temporal** workflow, turning inspectors' field notes into report inputs

Institute for Experiential AI, Northeastern University — Portland, ME
AI Engineer Co-op — *Aug 2025 – Feb 2026*
- Raised application-fraud (material misrepresentation) **Gini 0.26 → 0.40 (∼54% lift over production)** and first-decile capture 16% → 25%, replacing a PyTorch DNN with a **Transformer-encoder + XGBoost** ensemble (850K+ policies, 10 sources)
- Designed a stratified-sample validation, with underwriters, of a 120+ rule **Gemini 2.5 Pro** document-checking system (field extraction, cross-document checks), diagnosing per-rule failures as over-strict or over-permissive; the system cut review time **75% (8h → 2h per 100 applications)**
- Engineered **AWS Glue** ETL pipelines replicating 23M+ policy records (13 tables) into PostgreSQL on AWS RDS, with SQL source-to-target reconciliation reaching 99.97% data integrity
- Served the ensemble as a Dockerized Flask API on **AWS SageMaker** for quote-time scoring, load-tested at **p95 under 180 ms**

Indian Servers — Hyderabad, India
Machine Learning Engineer — *Sep 2023 – Aug 2024*
- Fine-tuned **DistilBERT** on 35K labeled e-commerce support tickets for customer-intent classification, improving accuracy 20%
- Built a hybrid routing pipeline combining intent classification, sentence-embedding similarity search, and FAQ/cache fallback; with async **Celery/Redis** inference, it cut latency 15% and bounce rate 35%

Projects

Contract Intelligence Service: Agentic Clause Extraction | GitHub | *FastAPI, pgvector, Claude API* — Sep 2026
- Built a clause-extraction service over CUAD (510 contracts, 13,101 labeled clauses): hybrid Postgres FTS + pgvector retrieval, a tool-calling agent with Pydantic outputs, verbatim grounding, and 30 deterministic rules, with OpenTelemetry and CI-gated evals
- Ablated retrieval over 84 cases and showed recall misleads for agents: skipping the cross-encoder reranker kept recall@20 at 0.917 with **∼360× lower latency**, yet cut end-to-end span F1 by 0.074 because the agent depends on rank order

Consumer Credit PD Modeling & Vintage Backtesting | GitHub | *PySpark, LightGBM, scikit-learn* — Aug 2026
- Built a champion–challenger PD framework on 1.3M+ LendingClub loans with an out-of-time split (train 2007–2015, test 2016–2018); a 37-feature LightGBM challenger beat a 3-feature logistic champion (**AUC 0.681 → 0.705, KS 0.266 → 0.299**)
- Engineered an origination-time PySpark feature pipeline from 2.26M raw records, excluding every post-origination field to control target leakage; caught FICO fields missing from the extract
- Showed both models' near-identical calibration drift came from the vintages, not the models: 90.5% of 2018 loans were still unresolved (label censoring), yet age-matched default curves showed 2018 loans genuinely defaulting slower

Technical Skills

Statistical Modeling: GLMs, Logistic Regression, XGBoost, LightGBM, SMOTE, Champion–Challenger, Vintage Backtesting
Validation: Gini, AUC, KS, PR-AUC, MAE, Calibration, Cost-Based Cutoffs, Decile Capture, Slice Analysis
Data & MLOps: Python, SQL (PostgreSQL, Spark SQL), PySpark, Databricks, MLflow, AWS Glue/RDS/SageMaker, Tableau
LLM & Apps: LoRA, Structured Extraction (Pydantic), Hybrid Retrieval, Reranking, vLLM, FastAPI, Docker, Temporal